Computer System Architecture and Parallel Computing

Fall 2026

Huan Wang, Westlake University

https://huanwang.tech

Lecture 7: Out-of-Order Execution & Tomasulo (ILP-2)

2026.10.19 Mon

Motivation: The Problem with In-Order Issue

In-order issue: issue follows program order; an unready head blocks everything behind it.
(a ready instruction cannot pass an unready predecessor — a direct limit on performance)

fld: Floating-point **L**oad **D**oubleword

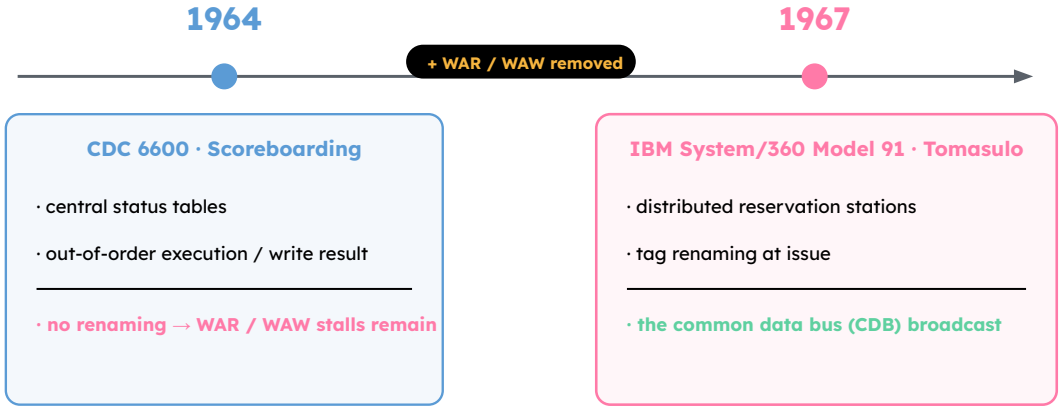
```
I1: fld    f0, 0(x1)     # cache miss, long latency
I2: fadd.d f4, f4, f0    # RAW — must wait for I1
I3: fadd.d f6, f8, f10   # fully independent of I1/I2
```

- **I1** — cache miss, long-latency wait
- **I2** — RAW through f0, waits for I1
- **I3** — operands ready, ALU free, yet **blocked behind**
- **Head-of-line blocking** — ready elements cannot pass an unready queue head
- Multiple issue allows **together**, never **passing**; in-order superscalar shares the limit

Btw, why use FP instructions? Integer ALU operations take one cycle, so an unready head costs little.

→ **We need out-of-order (OoO) execution or called dynamic (run-time) scheduling.**

Dynamic Scheduling: Scoreboarding → Tomasulo





CDC 6600 (delivered from 1964); the scoreboard was this machine's central control mechanism. Photo: Jitze Couperus, CC BY 2.0, via Wikimedia Commons
For scientific computing (FP, Multiply)

Historical note

The idea of OoO execution predated VLSI density (1978, DARPA, VLSI project) and classic 5-stage pipelining (1980s), and cache (1968): mainframe-only for decades, mainstream once transistor budgets allowed it

Ball back: 1964 was a landmark year for computer architecture: alongside the CDC 6600, IBM announced **System/360** (chief architect Gene Amdahl), defining the compatible-family ISA paradigm.

discrete component 分立原件
integrated circuits 集成电路

Hardware background of CDC6600

	Global lockstep pipeline (MIPS 5-stage) 1980s	CDC 6600 (scoreboard, independent FUs) 1964
Clock	One global clock; all pipeline registers update together	Shared clock, but no single chained pipeline; each FU runs independently
Progress	All instructions advance in lockstep; a stall freezes the whole pipeline	Each FU advances on its own; one blocked FU blocks no other
Structure	Fixed chain of stages: IF–ID–EX–MEM–WB	Parallel independent functional units — not a series pipe
Model	Hardware pipeline registers	Per-instruction state machine (Issue / Read / Execute / Write Result)

Refresh: The Five Pipeline Stages in Hardware



IF — Fetch
Read the instruction at PC from the instruction memory; an adder forms PC + 4.
Hardware: PC register, I-memory, adder

ID — Decode
Decode the opcode into control signals; read rs / rt from the register file; sign-extend the immediate.
Hardware: Control unit, register-file read ports, sign extender

EX — Execute
The ALU computes the result, the load / store address, or the branch condition and target.
Hardware: ALU, branch adder

MEM — Memory
A load reads, a store writes data memory at the EX address; other instructions pass through.
Hardware: D-memory read / write ports

WB — Write back
A multiplexer selects the ALU result or the loaded data and writes it to the destination register.
Hardware: Register-file write port, MUX

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) latch every signal at each clock edge — the “lockstep” the previous slide contrasts with the CDC 6600.

Diagram redrawn after the classic RISC pipeline figure — Patterson & Hennessy, Computer Organization and Design, ch. 4; https://en.wikipedia.org/wiki/Classic_RISC_pipeline

Scoreboarding: Issue Stage

What the Issue stage does (CDC 6600 scoreboard model):

1. Check: a functional unit is free; no WAW conflict with an in-flight instruction
2. Allocate a free functional unit
3. Add a row in the Instruction Status table, state = Issue
4. In the Unit Status table: FU.Busy = Yes; record Op, Fi (destination), Fj / Fk (sources)
5. Register Result Status table: RegisterResult[Fi] = this FU

In essence, issue **dispatches the instruction to an FU and registers it in the central scoreboard**, and the instruction then waits to read operands and execute. It moves from the fetch queue to the FU / scoreboard — **nothing is written to the architectural register file** yet.

Scoreboarding: Overview

Scoreboarding is a scheduling algorithm. The **scoreboard**: Maintain one central hardware table (3 subtables), answering each cycle — can it issue? are operands ready? can the result be written? It has 4 stages (not corresponding to 4 stages in pipelining).

Stage	Action	Stall when
1. Issue	allocate a free unit	structural (no free unit) · **WAW** in-flight writer (because of no renaming)
2. Read operands	read valid sources	**RAW** — a source is in flight
3. Execute	compute; may reorder	—
4. Write result	write via result bus, forward	**WAR** — an earlier reader needs the *old value* (because of no renaming)

```
I1: fadd f4, f0, f1
I2: fmul f4, f8, f10
```

Three tables

- Instruction status
- Functional unit (FU) status
- register result status

Achieved

In-order issue · **OoO execution and write result**; RAW via waiting + bus forwarding

Not achieved

Register names persist: **WAW blocks issue, WAR blocks writeback**

Scoreboarding routes around RAW waits at run time; **name dependences still block issue and writeback** —> Tomasulo algorithm (1967).

Scoreboarding — Animated Walkthrough

Scoreboarding — dynamic scheduling (CDC 6600 style)

cycle 1

Instruction status	Issue	Read operands	Execute	Write result
LD F6, 34(R2)	1			
LD F2, 45(R3)				
MULTD F0, F2, F4				
SUBD F8, F6, F2				
DIVD F10, F0, F6				
ADDD F6, F8, F2				

Functional-unit status

Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Load1	yes	LD	F6						
Load2	no								
Mult1	no								
Mult2	no								
Add	no								
Divide	no								

Register result status (who will write each register)

F0	F2	F4	F6	F8	F10
			Load1		

I1 issues to Load1. Register Result Status records: Load1 will write F6 (no other writer allowed).

The program, line by line

`LD F6, 34(R2)`: F6 = Mem[R2 + 34] — base register R2 + displacement 34 (bytes).

`LD F2, 45(R3)`: F2 = Mem[R3 + 45].

`MULTD F0, F2, F4`: F0 = F2 × F4 — the pattern is `OP Fi, Fj, Fk`, i.e. Fi = Fj op Fk.

F0–F10 are floating-point registers; R2/R3 are integer (address) registers.

Reading the tables: Fi = destination register; Fj/Fk = sources; Qj/Qk = the unit that will produce that operand (who to wait for); Rj/Rk = operand value ready and not yet read.

Watch I6 (ADDD): blocked at issue (Add busy — structural), then finished at cycle 12 but cannot write F6 until DIVD reads it (WAR, cycle 18).

Completion order **1, 2, 4, 3, 6, 5** — out-of-order, yet the final register file matches in-order execution.

How to read: every number is a clock cycle, counted from cycle 1 (top right of the frame). **Gold cells** mark exactly what changed this cycle — follow the gold to find each cycle's event. A value written at cycle c can be read at cycle c + 1.

Next: Tomasulo's Algorithm

So far: in-order issue has head-of-line blocking; scoreboarding executes out of order but still stalls on WAR and WAW name dependences.

Next: build the four components — reservation stations, renaming at issue, the common data bus, wakeup — then derive four instructions on the board.

Why it matters: these four components are the direct ancestors of the scheduler in every modern out-of-order core, including gem5's O3CPU in Lab 7.

Tomasulo: The Man, Two Principles, Two Terms

1967

1997 Eckert–Mauchly Award



Robert M. Tomasulo, IBM Computer Scientist, System/360 Model 91

The flagship scientific machine of the System/360 family; its **floating-point unit** ran the algorithm — built from discrete logic, **no compiler involved**.

Paper: *An Efficient Algorithm for Exploiting Multiple Arithmetic Units* (IBM J. R&D, 1967) [URL]

P1 — Hardware decides readiness

Not the compiler: an instruction executes **when its operands are ready and its functional unit is free** — not when its turn in program order comes

P2 — Register renaming

At issue, register names are replaced by **fresh tags**; WAR and WAW name dependences disappear, RAW survives as a tag chain

Term — Reservation station (RS)

A small waiting queue in front of each unit; every entry holds one issued instruction: the operation and each operand as a **value or a tag**

Term — Common data bus (CDB)

One broadcast channel: each finished result (tag + value) reaches **every waiting station and the register file in the same cycle**

Reservation Stations: Value or Provenance

One entry = all hardware state of one **issued, unfinished** instruction:

Field	Holds	Valid when
`busy`	entry occupied?	set at issue, cleared after write result
`Op`	operation (fadd / fmul…)	selects the unit class
`Vj, Vk`	source **values**	already available; copied at issue
`Qj, Qk`	producer **tags**	pending → non-empty; CDB match clears Q into V
`A`	effective address	load/store buffers only; once base is ready

Qi[f] — register result status

One tag per architectural register, pointing to the **station that will write f**; empty = stored value valid

Each operand slot: exactly one of value/tag

`Add1: fadd · Qj=L1 · Vk=f0 · busy=1`

Snapshot at issue

Operands are fixed from then on; later register rewrites **cannot affect a waiting station**

Each source operand answers one question: **is the value already here (in V), or must a tag be waited for (in Q)?** The instruction no longer remembers its original register names.

Renaming at Issue: Names Become Fresh Tags

The four-instruction board example: tag assignment at issue, cycle by cycle:

Cycle	Instruction	Station taken: operands recorded as	Qi after issue
c1	`I1: fld f2,0(x1)`	`L1: A=0(x1)`	`Qi[f2]=`**`L1`**
c2	`I2: fadd.d f4,f2,f0`	`Add1: Qj=`**`L1`**`, Vk=f0`	`Qi[f4]=`**`A1`**
c3	`I3: fmul.d f6,f8,f10`	`Mul1: Vj=`**`f8(old)`**`, Vk=f10`	`Qi[f6]=`**`M1`**
c4	`I4: fadd.d f8,f4,f6`	`Add2: Qj=`**`A1`**`, Qk=`**`M1`**	`Qi[f8]=`**`A2`**

WAR removed

I3 reads f8, I4 writes it: the old value is in Vj at c3; I4 gets fresh `A2` and **need not wait**

WAW removed

Same-name writers get **distinct tags**; Qi always points to the *latest*; no coupling through the name

RAW retained

True deps become **tag chains**: I2←L1, I4←A1/M1; produced values arrive via the CDB

Hardware register renaming: at issue, architectural names map to physical result locations named by tags. Name dependences arise from sharing one name; an exclusive name per in-flight instruction removes them.

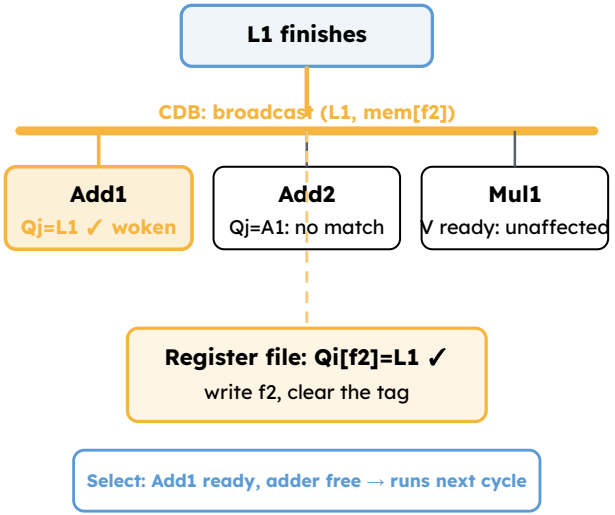
The Common Data Bus: One Broadcast, All Compare

(1) Broadcast
The finisher drives **(its tag, the result)**; one per cycle, others arbitrate

(2) Wakeup
All stations compare Qj/Qk *in parallel*; match → value into V, Q cleared — comparator hardware, not a lookup

(3) Select
Both Q empty and the unit free → candidate; arbitration picks who runs this cycle

(4) Register file, same cycle
Matching Qi → write value, clear tag; bus forwarding — no write-then-read wait



The CDB generalizes L3 forwarding: L3 bypassed *one* result to the *next* instruction's execute stage; the CDB broadcasts a result to *all* waiters and the register file at once.

The Three Phases

1

Issue — rename, park

Take the head; require a free station

Valid source → `V`; otherwise `Q=Qi[src]`

`Qi[dst] ← this tag`

Stall: no free entry (structural) — issue freezes

2

Execute — wait, compute

Both Q empty (or address A ready) → the unit computes

Before that, the instruction merely waits

RAW — await the producer's CDB broadcast; execution goes out of order

3

Write result — broadcast, free

Drive (tag, result) onto the CDB

Matching Q → copy value, clear Q; matching Qi → write register, clear tag

Free the station

Wait: CDB busy → arbitration, one per cycle

Terms: classic "issue" = enter station and rename, usually "dispatch" today; modern "issue" means wakeup/select out of the station (see the glossary)

No software involved: per-cycle decisions are comparator and arbitration circuits; the compiler only supplies the ISA program

The three constraint classes are each handled: structural hazards block at **issue**, RAW waits before **execute**, and WAR/WAW simply *do not exist* after renaming at issue.

Tomasulo's Algorithm — Animated Walkthrough

Tomasulo's algorithm — dynamic scheduling (IBM 360/91 style)

cycle 1

Instruction status	Issue	Execute	Write result
LD F6, 34(R2)	1		
LD F2, 45(R3)			
MULTD F0, F2, F4			
SUBD F8, F6, F2			
DIVD F10, F0, F6			
ADDD F6, F8, F2			

Reservation stations

Name	Busy	Op	Vj	Vk	Qj	Qk
Load1	yes	LD				
Load2	no					
Add1	no					
Add2	no					
Add3	no					
Mult1	no					
Mult2	no					

Register status (Qi — who will write each register)

F0	F2	F4	F6	F8	F10
			Load1		

I1 issues to Load1; renaming records Qi[F6] = Load1.

Same six instructions as the scoreboard animation — watch what is **missing**: no structural issue stalls, no WAR write stall.

Reservation stations hold each operand as a value (V) or a producer tag (Q); the register status Qi says who will write each register.

CDB broadcast: a finished result (tag + value) fills every matching Q in the same cycle — tags become values (wakeup).

Why no WAR stall: DIVD's source was renamed to a station tag at issue; ADDD writes the architectural F6 without harming anyone.

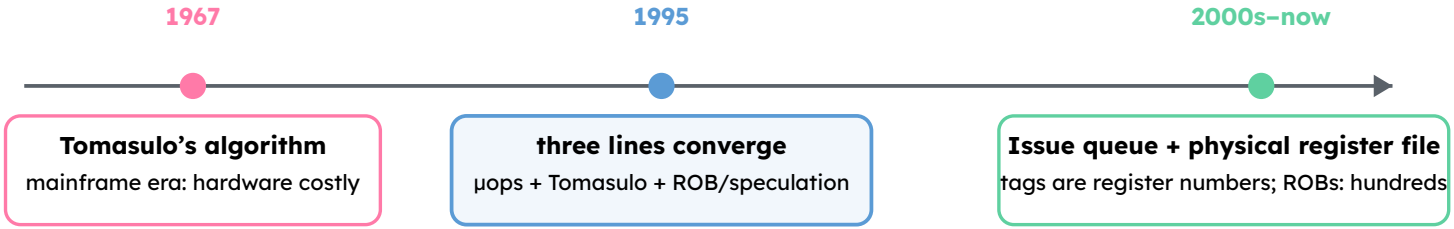
Example: Hennessy & Patterson, Computer Architecture: A Quantitative Approach — animation created for this course.

Summary: Scoreboarding vs. Tomasulo

Aspect	Scoreboard (CDC 6600, 1964)	Tomasulo (IBM 360/91, 1967)
Where status lives	One **central** scoreboard (three tables)	**Distributed** — in the reservation stations and the Qi register tags
Dependence tracking	Watches **register names**; polls operand readiness each cycle	**Tags (Qi) track producers**; the CDB **pushes** results by broadcast
Name dependences WAR/WAW	**Not eliminated** — WAW stalls issue, WAR stalls writeback	**Eliminated** — names are decoupled by renaming at issue
Result delivery	Wait for the **register-file writeback**, then read the register	CDB **bypass broadcast** — waiters receive the value in the broadcast cycle

Same goal — dynamic scheduling at run time. The difference is **distribution + renaming + broadcast**: the scoreboard's questions answered without a central table, without name-dependence stalls, and without waiting for writeback.

Onwards: From 1967 to Today



Historical lesson: ideas predate technology

1967: comparators and buffers too costly → mainframe-only for decades; 1990s: on-chip budgets allow it → standard in general-purpose cores

1995: three lines converge; the structure persists

µops (L6) + Tomasulo (L7) + ROB·prediction·speculation (L8) — this lecture builds the middle block

Equivalents in modern cores

reservation stations → **issue queue (scheduler)**; tags → **physical-register numbers**; register status table → **register alias table**

names change; the "value or provenance" wait logic does not

In-order cores did not disappear

Energy-first embedded and "little" cores still issue in order; microarchitecture is a cost–benefit choice, not a replacement

Summary: What Out-of-Order Buys and What It Costs

Gains

- tolerates latencies known only at run time (misses, bus occupancy, long ops)
- fills issue slots: one chain waits, independent chains use the units
- binary portability: wider windows in new cores, no recompilation

Costs

area and power:

- stations and comparators run in parallel every cycle;
- wakeup logic is on the frequency critical path design and verification complexity;
- speculation adds a microarchitectural side-channel surface (L8)

Measured magnitudes (typical general-purpose cores)

0.5–1 in-order cores

2–4 wide OoO cores · typical general code

6–8 peaks on ILP-rich regular code

Far below the issue-width peak: short blocks, RAW chains, misses and branches cap it; returns diminish with window size (Wall 1993; quantified in L8)

History: A Detour to Out-of-Order Execution

1964

CDC 6600 · scoreboard

First dynamic scheduling (S. Cray): ten heterogeneous functional units, out-of-order completion.

FP latencies differ by up to 40× — no fixed schedule can fit.

1967

IBM 360/91 · Tomasulo

Renaming tags + CDB broadcast: WAR/WAW eliminated by construction.

CAM compare and the broadcast bus are costly in discrete transistors — dynamic scheduling goes dormant.

1980s

RISC · five-stage pipeline

ISA co-designed with the pipeline: fixed format, load/store, single-cycle ALU.

In-order lockstep wins on clock rate and compiler-friendliness; out-of-order is shelved.

1990s

OoO returns · Tomasulo + ROB

Clock-rate wall: ILP becomes the only way forward.

Renaming + in-order commit via the reorder buffer — Pentium Pro (1995) makes out-of-order mainstream.

Today

Both live on

Five-stage pipeline: the teaching baseline and the embedded core.

OoO superscalar: every high-performance core — the 1964 idea, mainstream at last.

Lessons:

- Technologies come and go, but insights are forever.
- 技术的成败会随着条件变化 The success or failure of a tech is up to its condition.

Three Gaps Remain — All Closed in L8

1

Imprecise exceptions

Results reach registers out of order; at an exception “earlier done, later undone” cannot be guaranteed

Interrupts, page faults and traps require an ordered visible state

→ **L8: reorder buffer (ROB) + in-order commit**; speculative results can then be squashed together

2

The window stops at branches

Until the direction is known the window is confined to 4–7-instruction blocks

Branches are frequent; without crossing them a large window stays empty

→ **L8: branch prediction + speculative execution**; execute the predicted path, roll back through the ROB on a miss

3

Store→load RAW through memory

Whether addresses coincide is unknown until computed; a load cannot pass a store

Pointers make aliasing undecidable at compile time (the run-time counterpart of L6 GCD / restrict)

→ **L8: memory disambiguation**; loads speculatively pass stores, addresses checked afterward, violations replayed

With all three closed, classic Tomasulo becomes the **modern speculative out-of-order core**; L8 then asks how much ILP this actually mines — the answer is the ILP wall and the turn toward DLP/TLP.

Summary: Ready Instructions Go First

1 · Motivation: head-of-line blocking
Ready independent instructions are held behind an unready head

2 · Scoreboarding (1964): central tables, out-of-order execution
No renaming → WAR/WAW still stall

3 · Tomasulo (1967): three mechanisms
Stations wait distributed (value or tag) · **renaming at issue** removes WAR/WAW · the **CDB broadcast** wakes every waiter and forwards in the same step

4 · Three phases
Issue (rename into a station; structural hazards block here) → **execute** (wait for RAW tags; out of order) → **write result** (broadcast, free; out-of-order completion)

5 · Window value: fill waits with independent work; with non-blocking caches, MLP
RAW chains such as pointer chasing cannot be filled

In one sentence: **dynamic ILP uses tags at run time to track the real data flow, letting operand-ready instructions pass unready ones while preserving RAW order exactly**. L6 posed the problem, L7 built the engine, and L8 adds commit and speculation before reaching the ILP wall.

Further Reading & Tools

Textbook sections

Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* (6e) — **§3.4** (the CDC 6600 scoreboard), **§3.5** (Tomasulo: station fields, tag renaming, the CDB, the three phases, a full timing example), **§3.6** (ROB and speculation; detailed in L8)

Patterson & Hennessy, *Computer Organization and Design (RISC-V Edition)* — **§4.10** (an ILP-organized introduction; suitable as preparation)

Links and tools

gem5 documentation — official documentation for the Lab 7 simulator; MinorCPU in-order, O3CPU out-of-order

perf wiki and macOS Instruments — the IPC-measurement tools for the blog question (`perf stat -e cycles,instructions`)

Computer History Museum: Robert Tomasulo profile — historical background of the algorithm and the Model 91 context

Papers (every reference cited in this deck)

Thornton, *Parallel Operation in the Control Data 6600*, AFIPS FJCC 1964 — the original account of out-of-order execution and scoreboarding

Tomasulo, *An Efficient Algorithm for Exploiting Multiple Arithmetic Units*, IBM Systems Journal 1967 — first-hand definition of stations, tags and the CDB; 1997 Eckert–Mauchly Award

Smith & Pleszkun, *Implementing Precise Interrupts in Pipelined Processors*, IEEE TC 1988 — the source of the reorder-buffer idea

Yeager, *The MIPS R10000 Superscalar Microprocessor*, IEEE Micro 1996 — the designers' account of the first commercial out-of-order RISC processor

Gwennap, *Intel's P6 Uses Decoupled Superscalar Design*, Microprocessor Report 1995 — how x86 µops, Tomasulo-style scheduling and ROB speculation converged (slide 23)

Wall, *Limits of Instruction-Level Parallelism*, ASPLOS 1991 / WRL TR 1993 — quantitative basis for the s16 magnitudes; used systematically in L8

Appendix: Glossary (1/2)


```
Issue (enter an RS/buffer and rename) → [wait for operands as "value or tag"] → Execute → Write result (CDB broadcast)   [+ in-order Commit / ROB — L8]
```

Term	Definition	Notes / examples
Dynamic scheduling	Hardware decides cycle by cycle at run time when each instruction uses which functional unit; timing decisions are not fixed in machine code.	Counterpart to L6 compiler scheduling; scoreboarding and Tomasulo are both dynamic.
Head-of-line blocking	When the head of an ordered queue cannot be served, ready elements behind it cannot be served either.	Slide 4: a missing load and a RAW fadd block an independent fadd; out-of-order issue removes it.
Scoreboard	The CDC 6600's centralized dynamic scheduler: three tables (instruction status, unit status, register result status) answer whether to issue, read operands, or write results.	In-order issue, out-of-order execution and write result; no renaming — WAR stalls at write result, WAW at issue.
Architectural / physical register	Architectural registers are ISA-defined and named in encodings (f2, x1); physical registers are the actual result-holding cells, more numerous.	A tag is a physical result location temporarily provided by a station; in modern cores it is a physical register number.
Tag / Qi / V, Q fields	A tag is the unique hardware name of an in-flight instruction's result; Qi[f] names the station that will currently write register f; V holds a known operand value, Q the tag of its producer.	An empty Q means the operand is ready; slide 14: I2 has Qj=L1.
Reservation station / issue queue	A small hardware queue in front of a functional unit holding issued-but-not-started instructions; each entry holds Op, Vj/Vk, Qj/Qk, busy (memory buffers also A).	Classic term: reservation station; modern cores say issue queue or scheduler, same role.
Hardware register renaming	At issue, architectural names map to tag-named physical result locations, so every in-flight writer has a unique name.	Removes WAR/WAW; cannot remove RAW (kept as a tag chain).
Common data bus (CDB)	The shared bus broadcasting (producer tag, value) from a functional unit to all stations, memory buffers and the register file simultaneously.	Typically one result per cycle with arbitration; the generalization of L3 forwarding.
Wakeup / select	Two combinational steps per cycle: compare the broadcast tag against all Q fields in parallel (wakeup); arbitration picks among ready entries with free units (select).	Must complete in one clock cycle; dedicated comparator/arbitration circuits, not software.

Appendix: Glossary (2/2)

Term	Definition	Notes / examples
Issue / dispatch (terminology note)	In the classic paper “issue” means entering a station and renaming (used here); in modern pipelines this is usually “dispatch”, while “issue” means leaving the queue for a unit once operands are ready.	gem5/O3CPU documentation uses the modern usage; this lecture’s board work follows the classic three-phase wording.
In-/out-of-order issue, execution, completion	Issue, execution and completion (write result) may each follow program order or not; classic Tomasulo: issue in order, execution and completion out of order.	In the slide-14 walkthrough I3 executes and writes before I2.
Write result / writeback vs commit (retirement)	Write result: the result reaches waiters over the CDB and enters its (tag/physical) location, possibly out of order; commit: results become irrevocable architectural state in program order.	Classic Tomasulo has no commit step and writes registers directly (imprecise exceptions); commit is added by the L8 ROB.
Precise exception	A machine state at exception time in which all instructions before the faulting one have completed and taken effect, and none after it has.	Out-of-order architectural writes break it; in-order commit with a ROB restores it (L8).
Out-of-order / instruction window	At any instant, the set of issued-but-incomplete instructions among which hardware freely selects ready ones to execute.	In the classic model, occupied station and buffer entries; in modern cores, roughly the ROB capacity.
MLP (memory-level parallelism)	Multiple independent cache misses in flight at once; requires a non-blocking cache with MSHRs (L5) and a window that keeps independent loads in flight.	Pointer chasing is a RAW chain with one miss in flight; Lab 8 Task 8.0 models L=10.
µop (micro-op)	A RISC-like internal operation decoded from a CISC instruction (e.g. x86); Tomasulo-style scheduling works in µops.	The Pentium Pro (1995) fusion: µops + Tomasulo + ROB; defined also in the L6 glossary.